Available online at www.sciencedirect.com
ScienceDirect

Comput. Methods Appl. Mech. Engrg. 400 (2022) 115523

Computer methods in applied mechanics and engineering

www.elsevier.com/locate/cma

Monte Carlo fPINNs: Deep learning method for forward and inverse problems involving high dimensional fractional partial differential equations

Ling Guo[a], Hao Wu[b,c,*], Xiaochen Yu[c], Tao Zhou[d]

[a] *Department of Mathematics, Shanghai Normal University, Shanghai, China*
[b] *Institute of Natural Sciences and School of Mathematical Sciences, Shanghai Jiao Tong University, Shanghai, China*
[c] *School of Mathematical Sciences, Tongji University, Shanghai, China*
[d] *Institute of Computational Mathematics and Scientific/Engineering Computing, Academy of Mathematics and Systems Science, Chinese Academy of Sciences, Beijing, China*

Received 20 March 2022; received in revised form 15 July 2022; accepted 5 August 2022
Available online 16 September 2022

Abstract

We introduce a sampling-based machine learning approach, Monte Carlo fractional physics-informed neural networks (MC-fPINNs), for solving forward and inverse fractional partial differential equations (FPDEs). As a generalization of the physics-informed neural networks (PINNs), MC-fPINNs utilize a Monte Carlo approximation strategy to compute the fractional derivatives of the DNN outputs, and construct an unbiased estimation of the physical soft constraints in the loss function. Our sampling approach can yield lower overall computational cost compared to fPINNs proposed in Pang et al.(2019), hence it can solve high dimensional FPDEs at reasonable cost. We validate the performance of MC-fPINNs via several examples, including high dimensional integral fractional Laplacian equations, parametric identification of time–space fractional PDEs, and fractional diffusion equation with random inputs. The results show that MC-fPINNs are flexible and quite effective in tackling high dimensional FPDEs.
© 2022 Elsevier B.V. All rights reserved.

Keywords: Physics-informed neural networks; Fractional Laplacian; Nonlocal operators; Uncertainty quantification

1. Introduction

Fractional partial differential equations (FPDEs) have been employed in modeling systems involving memory and long-range interactions, such as solute transport in porous media [1], viscoelastic constitutive laws [2], and turbulent flow [3–5]. In practice, it is in general impossible to obtain analytical solutions for complex FPDEs and thus many numerical methods have been developed. Among others, we mention, for example, finite difference methods, finite element methods, and spectral methods. Readers are referred to [6,7] and reference therein for more details in this direction. The main challenge for numerically solving FPDEs is rooted in the high computational cost and high memory requirements due to the nonlocal property and singularity of the fractional derivatives. In particular, for

* Corresponding author.
E-mail address: hwu81@sjtu.edu.cn (H. Wu).

https://doi.org/10.1016/j.cma.2022.115523
0045-7825/© 2022 Elsevier B.V. All rights reserved.

inverse problems modeled by FPDEs, one needs to identify the fractional derivative order or other parameters from the observation data via expensive forward FPDEs solvers.

Recently, machine learning techniques have been developed to solve forward and inverse partial differential equations [8]. Among them are Gaussian process regression [9–16] and deep neural networks (DNNs) [17–26]. In this work, we focus on physics-informed neural networks (PINNs) that were first introduced in [20,27]. The key idea of PINNs is to include the physical law (i.e., the PDE) into a DNN that shares parameters with the DNN-surrogate for the solution of the PDE. This strategy enables one to use less data during the training process and can better express the physical law. We can thus predict the system state, unlike the deep learning approach driven solely by data. PINNs are simple to implement and easy for coding, and are successful for diverse forward and inverse problems in physics and fluid mechanics [28,29].

The success of DNN-based approaches for PDEs (such as PINNs) relies on well-developed tools such as automatic differentiation for dealing with integer-order partial differential equations. However, the automatic differentiation is not applicable to PDEs with nonlocal operators (such as FPDEs). To address this difficulty, in [30] the authors extended PINNs to fractional PINNs (fPINNs) for solving space–time fractional advection–diffusion equations. The main idea of fPINNs is to use automatic differentiation for the integer-order operators, while a numerical discretization scheme such as finite difference is employed for the fractional derivative of the neural network output. For example, the directional fractional Laplacian of the neural network output can be computed by combining the shifted vector Grünwald–Letnikov (GL) formula and quadrature rules. This approach leads to excessive cost as the physical dimension increases. Hence it is infeasible for solving high dimensional fractional PDEs. We also mention that a nonlocal PINN method for a parameterized nonlocal universal Laplacian operator has been investigated in [31].

In this paper, we propose a Monte Carlo sampling-based PINN, named MC-fPINNs, for solving forward and inverse fractional partial differential equations. The effectiveness of our MC-fPINNs lies in: (1) We compute the fractional derivative of the DNN output via Monte Carlo sampling. (2) During the training step, an unbiased estimate of the physics-based loss function is designed to obtain the optimal DNN parameters. Compared to fPINNs, our approach exhibits the following advantages:

- Unlike fPINNs, the fractional derivative of the DNN output is computed via a sampling approach instead of using traditional schemes such as the finite difference method, which alleviate the computational cost and is promising for high dimensional problems.
- The MC-fPINNs model can also be used for solving parametric FPDEs where the input parameters maybe random, e.g., in uncertainty quantification problems.

We demonstrate the effectiveness of MC-fPINNs by solving forward and inverse high dimensional space–time fractional advection–diffusion equations involving hyper-singular integral fractional Laplacian. Classical methods have been developed for 2D/3D forward fractional Laplacian problem in [32–37]. A mesh free method is also developed in [38] for high dimensional fractional Laplacian problem. While some works are focusing on 1D/2D inverse space–time fractional PDEs with a fractional Laplacian [39,40], less research has been conducted on the 3D problem. We consider in this work fractional Laplacian problems with dimensions up to 10 defined on a bounded domain, which are challenging for classical numerical methods.

The organization of this paper is as follows. In Section 2, we set up the forward and inverse FPDEs. In Section 3, we introduce a sampling approach to compute the fractional derivative of the DNN output, and this is followed by our main algorithm – the MC-fPINNs for solving fractional PDEs. In Section 4, we first present a detailed study of the accuracy and performance of our MC-fPINNs model for space fractional Laplacian operator in a bounded domain. Then, we present the simulation results for a space–time FPDE identification problem. Finally, we show the flexibility of MC-fPINNs in solving FPDEs with random inputs. We conclude the paper in Section 5.

2. Notations and problem setup

On a bounded spatial domain $\Omega \subset \mathbb{R}^d$, we consider the following high dimensional fractional advection–diffusion equation

$$\begin{aligned} \mathcal{L}[u(x,t)] := \tfrac{\partial^\gamma u(x,t)}{\partial t^\gamma} + c(-\Delta)^{\alpha/2}u(x,t) + v\cdot\nabla u(x,t) &= f(x,t), && (x,t)\in\Omega\times(0,T],\\ u(x,0) &= g(x), && x\in\Omega,\\ u(x,t) &= 0, && (x,t)\in\Omega^c\times(0,T], \end{aligned} \tag{1}$$

where c is the diffusion coefficient (deterministic or random variable), v is the mean-flow velocity, and we assume zero boundary condition for simplicity. Here $\frac{\partial^\gamma}{\partial t^\gamma}$ is the Caputo-type time-fractional derivatives of order γ defined by:

$$\frac{\partial^\gamma u(x,t)}{\partial t^\gamma} \triangleq \frac{1}{\Gamma(1-\alpha)} \int_0^t (t-\tau)^{-\gamma} \frac{\partial u(x,\tau)}{\partial \tau} d\tau, \quad 0 < \gamma < 1, \tag{2}$$

where $\Gamma(\cdot)$ is the Gamma function. The fractional Laplacian operator we considered in this paper is defined via the hyper-singular integral, i.e. [6]:

$$(-\Delta)^{\alpha/2} u(x) \triangleq C_{d,\alpha} \quad \text{P.V.} \int_{\mathbb{R}^d} \frac{u(x)-u(y)}{\|x-y\|_2^{d+\alpha}} \mathrm{d}y, \quad 0 < \alpha < 2, \tag{3}$$

where P.V. denotes the principle value of the integral and $C_{d,\alpha}$ is given by

$$C_{d,\alpha} = \frac{2^\alpha \Gamma(\frac{\alpha+d}{2})}{\pi^{d/2} |\Gamma(-\alpha/2)|}. \tag{4}$$

We consider two types of FPDE problems in this work:

- *Forward problem*: We know exactly the fractional-order α and γ, the diffusion coefficient c, the velocity v and the force term f, as well as the boundary/initial state to provide boundary/initial conditions of u, and our quantity of interest (QoI) is $u(x,t)$;
- *Inverse problem*: In addition to the boundary/initial conditions, we have a limited number of extra u-sensors that can be placed in the space–time domain $\Omega \times (0,T]$ to collect data, while we are interested in inferring α, γ, c, v and the entire information of the solution $u(x,t)$.

The main goal of this paper is to address both types of problems via a deep learning method. To this end, we construct a neural network surrogates $u_{NN}(x,t)$ for the solution $u(x,t)$ of (1) and then optimize the DNN parameters such that the approximation $u_{NN}(x,t)$ satisfies the observed data. Our main innovation is the formulation of unbiased estimation of the mean square equation loss function for the FPDEs, which results in dramatically reduced computational complexity (compared to fPINNs in [30]) and can be used to solve high dimensional FPDEs.

3. Methodology

3.1. Physics-Informed Neural Network (PINNs)

In this part, we first briefly review the main idea of DNNs-based approach for solving integer-order partial differential equations [17,18,27]. To this end, we consider the following one dimensional diffusion problem:

$$\begin{aligned} \mathcal{L}_\lambda[u(x,t)] := \tfrac{\partial u(x,t)}{\partial t} - \lambda \tfrac{\partial^2 u(x,t)}{\partial x^2} &= f(x,t), \quad (x,t) \in (0,1) \times (0,T], \\ u(x,0) &= g(x), \quad x \in (0,1), \\ u(0,t) = u(1,t) &= 0, \end{aligned} \tag{5}$$

where $u(x,t)$ is the solution and λ denotes the problem parameters.

The PINNs approach solve the above forward PDE problems via constructing a DNN surrogate $u_{NN}(x,t;\theta)$, parameterized by θ, of the solution $u(x,t)$. More precisely, u_{NN} takes the coordinate x,t as the input and outputs a vector that has the same dimension as $u(x,t)$. This surrogate u_{NN} is then substituted into Eq. (5) via automatic differentiation, which is conveniently integrated in many machine learning packages to obtain

$$f_{NN}(x,t;\theta) = \mathcal{L}_\lambda[u_{NN}(x,t;\theta)].$$

Assume that we have the training data set $\mathcal{D} = (\mathcal{D}_f, \mathcal{D}_g, \mathcal{D}_u)$, where

$$\mathcal{D}_f = \{x_i, t_i, f_i\}_{i=1}^{N_f}, \quad \mathcal{D}_g = \{x_i, 0, g_i\}_{i=1}^{N_g}, \quad \mathcal{D}_u = \{x_i, t_i, u_i\}_{i=1}^{N_u},$$

and

$$f_i = f(x_i,t_i), \quad g_i = g(x_i,t_i), \quad u_i = u(x_i,t_i).$$

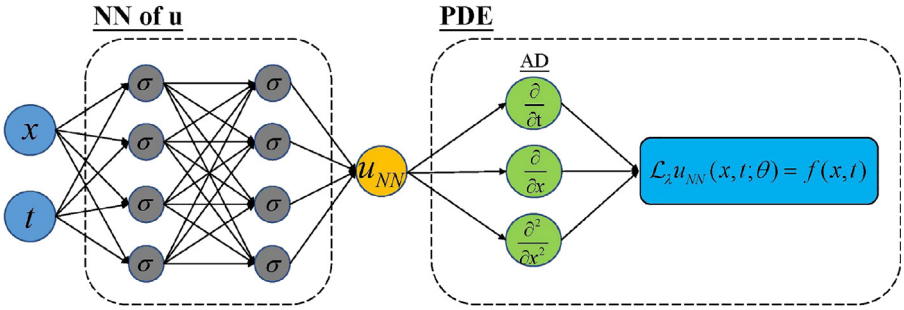

Fig. 1. Schematic of the PINNs for solving partial differential equations.

Notice that the data locations in the physical domain of f, g, and u are usually different in general. We use the same symbol (x_i, t_i) for simplicity here, since there is no misunderstanding. Then at the training stage, the DNN-parameters θ are optimized, denoted by $\hat{\theta}$, by fitting the data set $\mathcal{D} = (\mathcal{D}_f, \mathcal{D}_g, \mathcal{D}_u)$ via minimizing the following loss function:

$$\mathcal{LOSS}(\theta) = \frac{1}{N_u}\sum_{i=1}^{N_u}[u_{NN}(x_i, t_i; \theta) - u_i]^2 + \frac{1}{N_f}\sum_{i=1}^{N_f}[f_{NN}(x_i, t_i; \theta) - f_i]^2 + \frac{1}{N_g}\sum_{i=1}^{N_g}[u_{NN}(x_i, 0; \theta) - g_i]^2 . \quad (6)$$

Upon determination of $\hat{\theta}$, $u_{NN}(x, t; \hat{\theta})$ can be evaluated at any $(x, t) \in (0, 1) \times (0, T]$. Fig. 1 shows a sketch of the PINNs.

Inspired by the above PINNs approach, an extended version, named fractional PINNs (fPINNs), is established in [30] for solving fractional partial differential equations. However, automatic differentiation cannot be used directly for fractional derivatives. fPINNs thus employs the traditional discrete techniques, such as the finite difference method, for the fractional differential operators to obtain the loss function of PINNs. Obviously, this approach needs many auxiliary points for each training point and thus suffers from the curse of dimensionality for high dimensional problems. To tackle this problem, we shall employ a Monte Carlo approximation strategy to compute the fractional derivatives of the DNN outputs and establish our MC-fPINNs strategy for solving forward and inverse problems of fractional PDEs.

3.2. Monte Carlo fractional Physics-Informed Neural Networks (MC-fPINNs)

In this section, we formalize the algorithm for solving Eq. (1). Given the training data set $\mathcal{D} = (\mathcal{D}_f, \mathcal{D}_g, \mathcal{D}_u)$, we define the loss function as

$$\mathcal{LOSS}(\theta) = w_{\text{equ}} L_{\text{equ}}(\theta) + w_g L_g(\theta) + w_u L_u(\theta), \quad (7)$$

where

$$L_{\text{equ}}(\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f}[\mathcal{L}[u_{NN}(x_i, t_i; \theta)] - f(x_i, t_i)]^2, \quad L_g(\theta) = \frac{1}{N_g}\sum_{i=1}^{N_g}[u_{NN}(x_i, 0; \theta) - g_i]^2,$$
$$L_u(\theta) = \frac{1}{N_u}\sum_{i=1}^{N_u}[u_{NN}(x_i, t_i; \theta) - u_i]^2. \quad (8)$$

Here w_{equ}, w_g and w_u are the weights of the different parts of the loss function. As in the training of common PINNs, the minimization of the loss function can be carried out by stochastic gradient descent (SGD). Therefore, it is actually unnecessary to obtain the accurate approximation of the involved nonlocal operators, including $\frac{\partial^\gamma}{\partial t^\gamma}$ and $(-\Delta)^{\alpha/2}$, and parameters θ can be updated with an unbiased estimate of the gradient of $\mathcal{LOSS}$. The proposed MC-fPINNs employs this idea as shown in Fig. 2. It approximates the nonlocal operators in $\mathcal{L}$ by a Monte Carlo method so that SGD can reach the local minimum. In the next section, we shall give details of the Monte Carlo approximation.

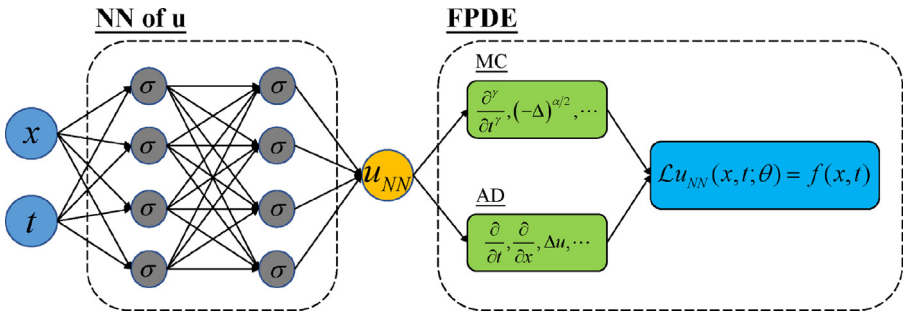

Fig. 2. Schematic of the MC-fPINNs for solving forward and inverse fractional partial differential equations.

Remark 3.1. The convergence of SGD to local minima and the convergence rate with unbiased estimates of gradients are beyond the scope of this paper, and have been thoroughly studied in the field of machine learning (see, e.g., [41,42]).

3.2.1. Monte Carlo approximation of fractional operators

To compute the fractional Laplacian of the DNN output $u_{NN}(x,t;\theta)$ with $\alpha \in (0,2)$, we first divide the integral into integrals over a neighborhood $B_{r_0}(x) = \{y \mid \|y-x\| \le r_0\}$ around x and its complement as the following:

$$(-\Delta)^{\alpha/2} u_{NN}(x) = C_{d,\alpha}\left(\int_{y\in B_{r_0}(x)} \frac{u_{NN}(x)-u_{NN}(y)}{\|x-y\|_2^{d+\alpha}}\mathrm{d}y + \int_{y\notin B_{r_0}(x)} \frac{u_{NN}(x)-u_{NN}(y)}{\|x-y\|_2^{d+\alpha}}\mathrm{d}y\right). \tag{9}$$

Here we omit t and θ and denote $u_{NN}(x,t;\theta)$ as $u_{NN}(x)$ for notation simplicity.

We can re-write the first part as

$$\begin{aligned}
\int_{y\in B_{r_0}(x)} \frac{u_{NN}(x)-u_{NN}(y)}{\|x-y\|_2^{d+\alpha}}\mathrm{d}y &= \int_{\|y\|_2\le r_0} \frac{u_{NN}(x)-u_{NN}(x-y)}{\|y\|_2^{d+\alpha}}\mathrm{d}y \\
&= \frac{1}{2}\int_{\|y\|_2\le r_0} \frac{2u_{NN}(x)-u_{NN}(x-y)-u_{NN}(x+y)}{\|y\|_2^{d+\alpha}}\mathrm{d}y \\
&= \frac{1}{2}\int_{S^{d-1}}\int_0^{r_0} \frac{2u_{NN}(x)-u_{NN}(x-r_I\xi)-u_{NN}(x+r_I\xi)}{r_I^{1+\alpha}}\mathrm{d}r_I\mathrm{d}\xi \\
&= \frac{1}{2}\int_{S^{d-1}}\int_0^{r_0} \frac{2u_{NN}(x)-u_{NN}(x-r_I\xi)-u_{NN}(x+r_I\xi)}{r_I^{2}}r_I^{1-\alpha}\mathrm{d}r_I\mathrm{d}\xi \\
&= \frac{\left|S^{d-1}\right| r_0^{2-\alpha}}{2\,(2-\alpha)}\mathbb{E}_{\xi,r_I\sim f_I(r_I)}\left[\frac{2u_{NN}(x)-u_{NN}(x-r_I\xi)-u_{NN}(x+r_I\xi)}{r_I^2}\right],
\end{aligned} \tag{10}$$

where ξ is uniformly distributed on the unit $(d-1)$-sphere S^{d-1}, $|S^{d-1}|$ denotes the surface area of S^{d-1},

$$f_I(r_I) = \frac{2-\alpha}{r_0^{2-\alpha}} r_I^{1-\alpha}\cdot 1_{r_I\in[0,r_0]},$$

and r_I can be sampled as

$$r_I/r_0 \sim \mathrm{Beta}(2-\alpha, 1). \tag{11}$$

Notice that we have

$$\lim_{r\to 0}\frac{2u_{NN}(x)-u_{NN}(x-r_I\xi)-u_{NN}(x+r_I\xi)}{r_I^2} = \left.\frac{\partial^2 u_{NN}(x+r_I\xi)}{\partial r_I^2}\right|_{r_I=0},$$

and this finite-difference type approximation may suffer from the rounding error and yield numerical instability for an extremely small r. Thus, we utilize the following approximation of (10) in practice

$$\int_{y\in B_{r_0}(x)} \frac{u_{NN}(x)-u_{NN}(y)}{\|x-y\|_2^{d+\alpha}}\mathrm{d}y \approx \frac{\left|S^{d-1}\right| r_0^{2-\alpha}}{2\,(2-\alpha)}\mathbb{E}_{\xi,r_I\sim f_I(r_I)}\left[\frac{2u_{NN}(x)-u_{NN}(x-r_\epsilon\xi)-u_{NN}(x+r_\epsilon\xi)}{r_\epsilon^2}\right],$$

with $r_\epsilon = \max\{\epsilon, r_I\}$, where $\epsilon > 0$ is a small positive number.

Similarly, for the second part in (9) we have

$$\begin{aligned}\int_{y\notin B_{r_0}(x)} \frac{u_{NN}(x)-u_{NN}(y)}{\|x-y\|_2^{d+\alpha}}\mathrm{d}y &= \int_{\|y\|_2\geq r_0} \frac{u_{NN}(x)-u_{NN}(x-y)}{\|y\|_2^{d+\alpha}}\mathrm{d}y \\ &= \int_{S^{d-1}}\int_{r_0}^{\infty} \frac{u_{NN}(x)-u_{NN}(x-r_o\xi)}{r_o^{1+\alpha}}\mathrm{d}r_o\mathrm{d}\xi \\ &= \frac{\left|S^{d-1}\right| r_0^{-\alpha}}{2\alpha}\mathbb{E}_{\xi,r_o\sim f_O(r_o)}\left[2u_{NN}(x)-u_{NN}(x-r_o\xi)-u_{NN}(x+r_o\xi)\right],\end{aligned} \tag{12}$$

where

$$f_O(r_o) = \alpha r_0^{\alpha} r_o^{-1-\alpha} 1_{r_o\in[r_0,\infty)},$$

and r_o can be sampled via

$$r_0/r_o \sim \mathrm{Beta}(\alpha, 1). \tag{13}$$

Consequently, by combining (10) and (12), the fractional Laplacian of the surrogate u_{NN} can be calculated via the following approximation:

$$\begin{aligned}(-\Delta)^{\alpha/2}u_{NN}(x) \approx\ & C_{d,\alpha}\frac{\left|S^{d-1}\right| r_0^{2-\alpha}}{2\,(2-\alpha)}\mathbb{E}_{\xi,r_I\sim f_I(r_I)}\left[\frac{2u_{NN}(x)-u_{NN}(x-r_\epsilon\xi)-u_{NN}(x+r_\epsilon\xi)}{r_\epsilon^2}\right] \\ &+ C_{d,\alpha}\frac{\left|S^{d-1}\right| r_0^{-\alpha}}{2\alpha}\mathbb{E}_{\xi,r_o\sim f_O(r_o)}\left[2u_{NN}(x)-u_{NN}(x-r_o\xi)-u_{NN}(x+r_o\xi)\right].\end{aligned} \tag{14}$$

Here $r_\epsilon = \max\{\epsilon, r_I\}$, and r_I is sampled according to Eq. (11) while r_o is sampled via Eq. (13).

To approximate the time fractional derivative of the DNN output $u_{NN}(x,t;\theta)$ in Eq. (1) for $\gamma \in (0,1)$, we adopt again the Monte Carlo approximation via sampling as follows:

$$\begin{aligned}\frac{\partial^\gamma u_{NN}(x,t)}{\partial t^\gamma} &= \int_0^t (t-\tau)^{-\gamma}\frac{\partial}{\partial\tau}u_{NN}(x,\tau)\mathrm{d}\tau \\ &= \gamma\int_0^t \tau^{-\gamma}\frac{u_{NN}(x,t)-u_{NN}(x,t-\tau)}{\tau}\mathrm{d}\tau + \frac{u_{NN}(x,t)-u_{NN}(x,0)}{t^\gamma} \\ &= \frac{\gamma}{(1-\gamma)}t^{1-\gamma}\mathbb{E}_{\tau\sim f_{I,t}}\left[\frac{u_{NN}(x,t)-u_{NN}(x,t-\tau)}{\tau}\right] + \frac{u_{NN}(x,t)-u_{NN}(x,0)}{t^\gamma} \\ &\approx \frac{\gamma}{(1-\gamma)}t^{1-\gamma}\mathbb{E}_{\tau\sim f_{I,t}}\left[\frac{u_{NN}(x,t)-u_{NN}(x,t-\tau_\epsilon t)}{\tau_\epsilon t}\right] + \frac{u_{NN}(x,t)-u_{NN}(x,0)}{t^\gamma},\end{aligned} \tag{15}$$

where $f_{I,t}(\tau) = (1-\gamma)\tau^{-\gamma}\cdot 1_{\tau\in[0,1]}$, and τ can be sampled via

$$\tau \sim \mathrm{Beta}(1-\gamma, 1). \tag{16}$$

Moreover, $\tau_\epsilon = \max\{\tau, \epsilon_t t^{-1}\}$, and ϵ_t is a small positive number which we will specify it in the numerical examples.

Based on the above Monte Carlo approximations for the space and time-fractional derivative (14) and (15), along with the automatic differentiation for the integer-order derivative, we can finally obtain the approximation for $L[u_{NN}(x,t;\theta)]$ as

$$\mathcal{L}[u_{NN}(x,t;\theta)] \approx \mathbb{E}\left[\widehat{\mathcal{L}}[u_{NN}(x,t;\theta);\epsilon,\epsilon_t,\tau,\xi,r_I,r_o]\right] \tag{17}$$

with

$$\begin{aligned}\widehat{\mathcal{L}}[u_{NN}(x,t;\theta);\epsilon,\epsilon_t,\tau,\xi,r_I,r_o] \triangleq & \frac{\gamma t^{1-\gamma}}{(1-\gamma)}\cdot\frac{u_{NN}(x,t;\theta)-u_{NN}(x,t-\tau_\epsilon t;\theta)}{\tau_\epsilon t}\\ &+\frac{u_{NN}(x,t;\theta)-u_{NN}(x,0;\theta)}{t^\gamma}\\ &+\frac{C_{d,\alpha}\left|S^{d-1}\right|r_0^{2-\alpha}}{2(2-\alpha)}\\ &\cdot\frac{2u_{NN}(x,t;\theta)-u_{NN}(x-r_\epsilon\xi,t;\theta)-u_{NN}(x+r_\epsilon\xi,t;\theta)}{r_\epsilon^2}\\ &+\frac{C_{d,\alpha}\left|S^{d-1}\right|r_0^{-\alpha}}{2\alpha}\cdot(2u_{NN}(x,t;\theta)-u_{NN}(x-r_o\xi,t;\theta)\\ &-u_{NN}(x+r_o\xi,t;\theta))\\ &+v\cdot\nabla u_{NN}(x,t;\theta),\end{aligned} \tag{18}$$

Here r_I, r_o, τ are sampled according to f_I, f_O, $f_{I,t}$, $r_\epsilon = \max\{\epsilon, r_I\}$, $\tau_\epsilon = \max\{\tau, \epsilon_t t^{-1}\}$, and ξ is drawn from the uniform distribution on the sphere S^{d-1}. ϵ and ϵ_t are small positive numbers that we will specify later in the numerical simulations. The equality in (17) holds exactly if $\epsilon = \epsilon_t = 0$.

Remark 3.2. The last term on r.h.s of Eq. (18), which involves the integer-order derivative only, can be equivalently written as

$$v\cdot\nabla u_{NN}(x,t;\theta) = \frac{\partial u_{NN}(x+s\cdot v,t;\theta)}{\partial s}|_{s=0},$$

with s being a scalar. Therefore, we can calculate this term and its gradient with respect to θ by the automatic differentiation with $\mathcal{O}(1)$ passes through each layer of u_{NN}, which can be efficiently implemented even for a high dimensional domain Ω [43].

3.2.2. Unbiased estimation of the equation loss

We now describe how to evaluate the unbiased estimates of the equation loss L_{equ} in Eq. (7). Based on the Monte Carlo approximation for the fractional PDE operator Eq. (18), we can obtain the unbiased estimates of L_{equ} by implementing the following steps:

1. Select x_i and t_i from $\Omega \subset \mathbb{R}^d$ and the time domain, respectively, $i = 1, \ldots, N_f$.
2. Given small numbers ϵ and ϵ_t, for each residual point (x_i, t_i), we sample two groups random parameters τ, ξ, r_I, r_o according to their prior distributions and denote them by $\{\tau_i, \xi_i, r_{Ii}, r_{oi}\}_{i=1}^m$, $\{\tau_i', \xi_i', r_{I_i}', r_{o_i}'\}_{i=1}^m$ respectively. Here m represents the number of samples.
3. Then we can obtain two independent Monte Carlo estimates of $\mathcal{L}[u_{NN}(x_i,t_i;\theta)]$ as

$$\mathcal{L}_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] = \frac{1}{m}\sum_{j=1}^m \widehat{\mathcal{L}}[u_{NN}(x_i,t_i;\theta);\epsilon,\epsilon_t,\tau_j,\xi_j,r_{Ij},r_{oj}],$$

$$\mathcal{L}'_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] = \frac{1}{m}\sum_{j=1}^m \widehat{\mathcal{L}}[u_{NN}(x_i,t_i;\theta);\epsilon,\epsilon_t,\tau_j',\xi_j',r_{Ij}',r_{oj}']$$

according to Eq. (18), which lead to the following approximation of L_{equ}:

$$\hat{L}_{\text{equ}}(\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f}[\mathcal{L}_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] - f(x_i,t_i)]\cdot\left[\mathcal{L}'_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] - f(x_i,t_i)\right]. \tag{19}$$

According to the above analysis, we show in Appendix A that unbiased estimates of both $L_{\text{equ}}(\theta)$ and its gradient can be obtained from Eq. (19), i.e.,

$$\mathbb{E}[\hat{L}_{\text{equ}}(\theta)] = L_{\text{equ}}(\theta), \quad \mathbb{E}\left[\frac{\partial}{\partial\theta}\hat{L}_{\text{equ}}(\theta)\right] = \frac{\partial}{\partial\theta}L_{\text{equ}}(\theta) \tag{20}$$

if $\epsilon = \epsilon_t = 0$ and the rounding error can be ignored. Now we are ready to substitute Eq. (19) into Eq. (7) to formulate the total loss function. We summarize the MC-fPINN method in Algorithm 1.

Remark 3.3. One can also approximate L_{equ} and $\frac{\partial}{\partial\theta}L_{\text{equ}}$ by directly replacing $\mathcal{L}[u_{NN}]$ with $\mathcal{L}_{\text{MC}}[u_{NN}]$ as

$$\begin{aligned} L_{\text{equ}}(\theta) &\approx \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathcal{L}_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] - f(x_i,t_i)\right]^2, \\ \frac{\partial}{\partial\theta}L_{\text{equ}}(\theta) &\approx \frac{\partial}{\partial\theta}\left(\frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathcal{L}_{\text{MC}}[u_{NN}(x_i,t_i;\theta)] - f(x_i,t_i)\right]^2\right). \end{aligned} \tag{21}$$

The estimates are consistent as $m \to \infty$, but the unbiasedness for a finite m does not hold in general (see Appendix A).

Algorithm 1 MC-fPINNs for forward and inverse fractional PDEs

- **1.** Specify the training data set

$$\mathcal{D} = (\mathcal{D}_f, \mathcal{D}_g, \mathcal{D}_u), \text{ where } \mathcal{D}_f = \{x_i, t_i, f_i\}_{i=1}^{N_f}, \mathcal{D}_g = \{x_i, t_i, g_i\}_{i=1}^{N_g}, \mathcal{D}_u = \{x_i, t_i, u_i\}_{i=1}^{N_u}$$

- **2.** Sample N snapshots from the above training data
- **3.** Calculate the loss $\mathcal{LOSS}(\theta) = w_{\text{equ}}\hat{L}_{\text{equ}}(\theta) + w_g L_g(\theta) + w_u L_u(\theta)$ via Eqs. (7) and (19)
- **4.** Let $\theta \leftarrow \text{Adam}(\theta - \eta\frac{\partial\mathcal{LOSS}(\theta)}{\partial\theta})$ to update all the involved parameters in (7), η is the learning rate
- **5.** Repeat **Step 2-4** until convergence

4. Simulation results

This section consists of three parts, which address the two types of data-driven problems set up in the introduction. We first investigate the performance of the MC-fPINN method for solving high dimensional fractional Poisson problems. Then we demonstrate the efficiency of the MC-fPINN method to solve inverse fractional advection–diffusion equations. Subsequently, we shall consider solving an uncertainty quantification problem of FPDEs with unknown parameters.

In all our computations, we consider the relative l_2 error of the solution predicted by MC-fPINNs:

$$\text{Relative } l_2 \text{ error} = \frac{\|u_{NN}(x,t) - u(x,t)\|}{\|u(x,t)\|}, \tag{22}$$

where u and u_{NN} are the fabricated and surrogate solutions, respectively. We set $\epsilon = 10^{-3}$ and $\epsilon_t = 10^{-6}$ in the implementation for the Monte Carlo approximation of the equation loss, and choose 1000 test points uniformly in the physical domain to compute the relative error. Unless stated otherwise, the DNNs model contains four hidden layers, with 64 neurons per hidden layer.

4.1. High dimensional fractional Poisson problem

We start with the following fractional Poisson problem

$$(-\Delta)^{\alpha/2}u(x) = f(x), \quad x \in \Omega = \{x \mid \|x\|_2^2 \le 1\} \subset \mathbb{R}^d \tag{23}$$

with zero boundary conditions. A manufactured solution is chosen to be

$$u(x) = \left(1 - \|x\|_2^2\right)_+^{1+\alpha/2}, \quad \forall\alpha \in (0,2),$$

here $x_+ = \max\{x, 0\}$, and the corresponding forcing term is given by [44]

$$f(x) = 2^\alpha\Gamma\left(\frac{\alpha}{2}+2\right)\Gamma\left(\frac{\alpha+d}{2}\right)\Gamma\left(\frac{d}{2}\right)^{-1}\left(1 - \left(1+\frac{\alpha}{d}\right)\|x\|_2^2\right).$$

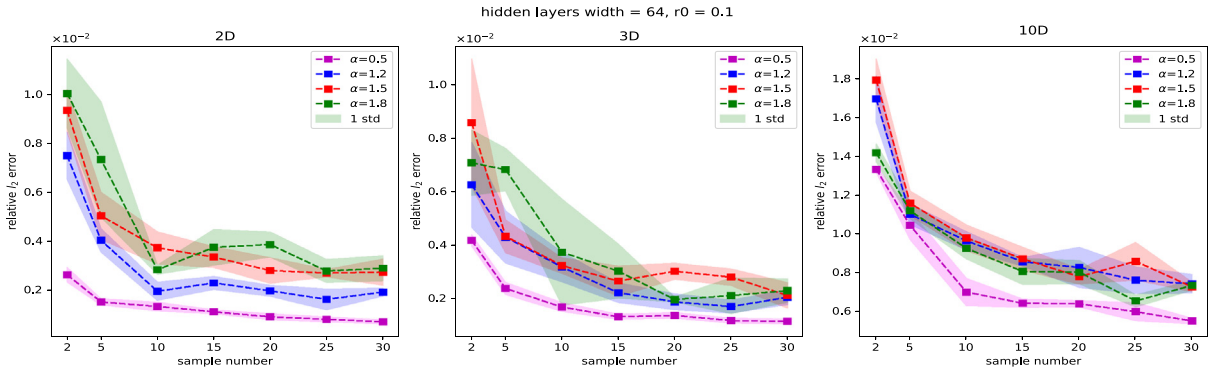

Fig. 3. Convergence for a fabricated solution $u(x) = \left(1 - \|x\|_2^2\right)_+^{1+\alpha/2}$. Relative l_2 error versus the parameter sample number for fixed number of r_0. **Left:** 2D; **Middle:** 3D; **Right:** 10D. The colored lines and shaded regions correspond to mean values and one-standard-deviation bands of relative l_2 errors of the MC-fPINN method, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

We approximate $u(x,t)$ with $u_{NN}(x,t) = (1 - \|x\|_2^2)_+ \cdot \tilde{u}_{NN}(x,t)$ in the simulations. Thus, we do not need to place training points on the boundary, since $u_{NN}(x,t)$ satisfies the boundary conditions automatically. To ensure the best performance of the MC-fPINNs, we use Adam optimizer with changing learning rate up to $M = 10^4$ iterations. The number of residual points used for computing the equation loss for each mini-batch is taken as 128, and we repeat drawing the residual points independently from the uniform distribution on Ω for each iteration, which implies that the equation loss

$$L_{\text{equ}} = \frac{1}{|\Omega|} \int_{\Omega} \left((-\Delta)^{\frac{\alpha}{2}} u(x) - f(x) \right)^2 \mathrm{d}x$$

is minimized during the training procedure.

We first report the impact of the sample number m of random instrumental variables in (19) and the neighborhood radius r_0 in (9), which are used in the Monte Carlo approximation of the fractional operators, on the accuracy of the MC-fPINN method for different fractional orders $\alpha = 0.5$, 1.2, 1.5 and 1.8 respectively. We run the MC-fPINNs code five times for each fractional order and plot the mean and one-standard-deviation band for the relative l_2 errors in Figs. 3 and 4. We can see that the error decays a little within a magnitude with increasing sample number when fixed r_0 and in the meantime, the error is getting smaller, as r_0 increases with fixed sample number. We can also see that the uncertainty is decreasing for a larger sample number and r_0. But finally, the error saturates around 10^{-3}, which shows that the choice for the sample number and r_0 do not have a significant impact on the accuracy for given α.

Based on the above results on the relative error depending on r_0 and the sample number, next, we show the accuracy of the MC-fPINN method for 2D and 3D fractional Laplacians with $\alpha = 1.5$, respectively. We run the MC-fPINNs code five times with fixed sample numbers 25 and $r_0 = 0.2$ and plot the mean prediction. Fig. 5 displays the contour plots of the fabricated solutions, MC-fPINNs predict solutions, and the absolute errors of the solutions in comparison with the fabricated solutions, respectively. We can see that the error is around 10^{-3}, which is sufficiently low, especially for a high dimensional problem.

We now compare the efficiency of MC-fPINNs and fPINNs [30] for solving fractional Poisson problems. To calculate the fractional Laplacian $(-\Delta)^{\alpha/2}$ at a given point x, fPINN discretizes the sphere S^{d-1} into $\mathcal{O}(q^{d-1})$ differentiation directions based on the polar coordinate system,[1] and utilize the shifted vector Grüwald–Letnikov (GL) formula to approximate the fractional derivative along each direction by $\mathcal{O}(L)$ auxiliary points, where q denotes the number of quadrature points in each polar coordinate and $1/L$ is the average spatial step size. While by using the Monte Carlo calculation of the fractional Laplacian in the proposed MC-fPINN, we only need to evaluate u_{NN} at $8m + 1$ points to obtain the unbiased estimate of the equation loss at x according to (18) and (19). Therefore,

[1] More specifically, 2 differentiation directions for $d = 1$ and q^{d-1} for $d > 1$.

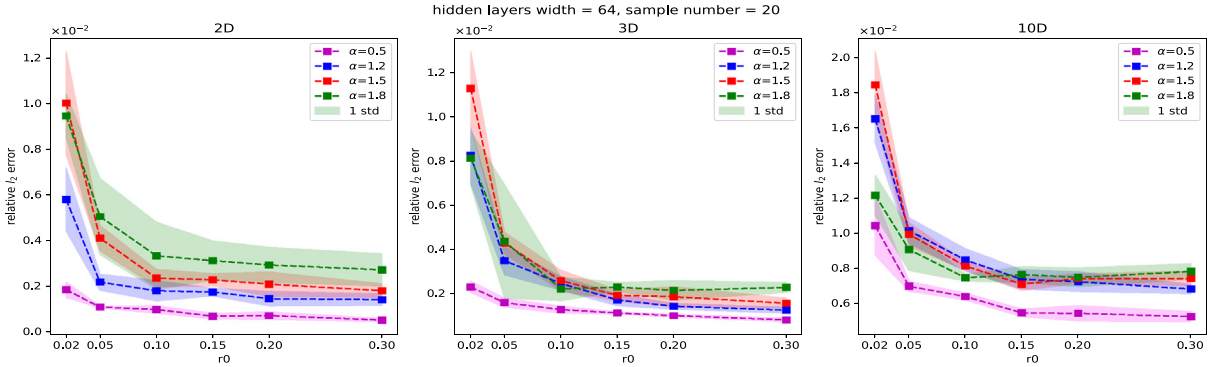

Fig. 4. Convergence for a fabricated solution $u(x) = \left(1 - \|x\|_2^2\right)_+^{1+\alpha/2}$. Relative l_2 error versus the parameter r_0 with fixed sample number 20. **Left:** 2D; **Middle:** 3D; **Right:** 10D. **Right:** 10D. The colored lines and shaded regions correspond to mean values and one-standard-deviation bands of relative l_2 error of the MC-fPINN method, respectively. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

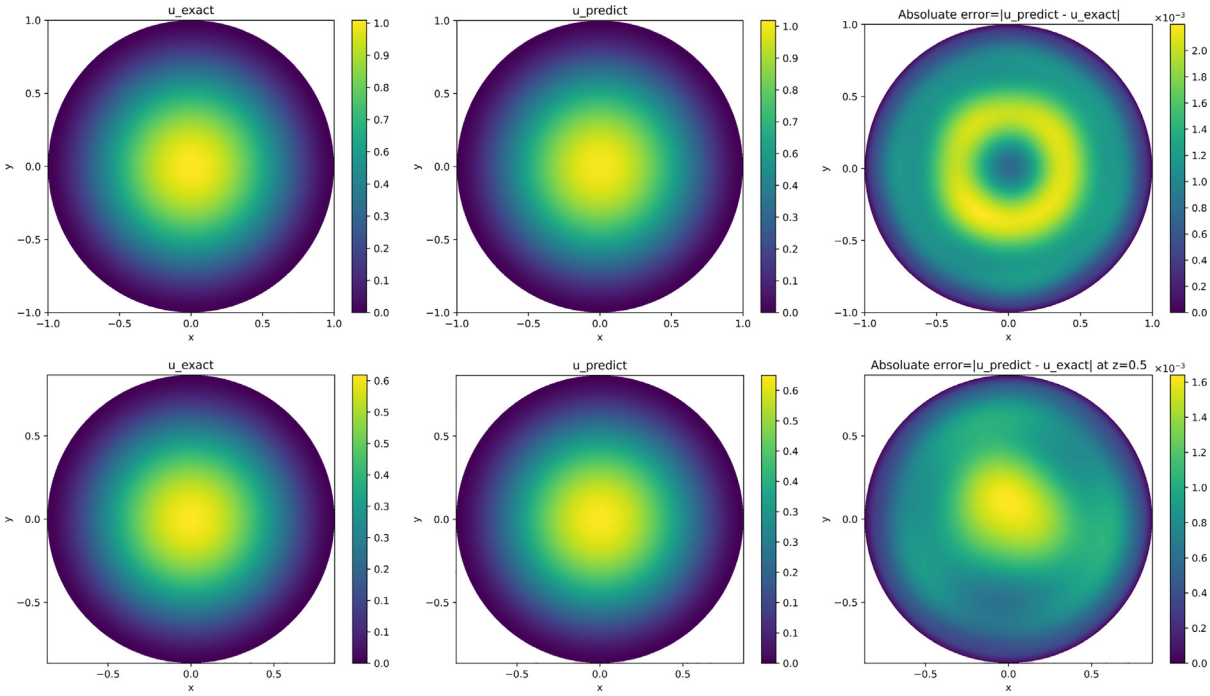

Fig. 5. MC-fPINNs accuracy in multidimensional simulations of fractional Laplacian with the fabricated solutions $u(x) = \left(1 - \|x\|_2^2\right)^{1+\alpha/2}$. **Top:** 2D exact solution (left), the MC-fPINNs solution (middle) and the corresponding absolute error (right). **Bottom:** 3D exact solution (left), the MC-fPINNs solution (middle) and the corresponding absolute error at $x_3 = 0.5$ (right).

for an SGD based training process with N_f training points and M iterations, the evaluation times of the surrogate model u_{NN} in fPINN and MC-fPINN are

$$\begin{aligned} \text{fPINN:} \quad & \mathcal{O}(q^{d-1} L M N_f), \quad \text{where } q = 8 \sim 15,\ L = 100 \sim 1000 \text{ for } d \le 3 \text{ in [30]} \\ \text{MC-fPINN:} \quad & (8m+1) M N_f, \quad \text{where } m = 20 \text{ for } d \le 10 \text{ in our experiments} \end{aligned}$$

We also plot the relative l_2 error versus number of u_{NN} that we need to calculate during the training process in Fig. 6 for the 2D and 3D fractional Poisson problems, where fPINN is implemented by the DeepXDE package [45]. It can be observed that MC-fPINN achieves small relative errors much faster than fPINN since MC-fPINN can update trainable parameters with smaller numbers of evaluation times of u_{NN} at each iteration. For $d \ge 4$, fPINN becomes

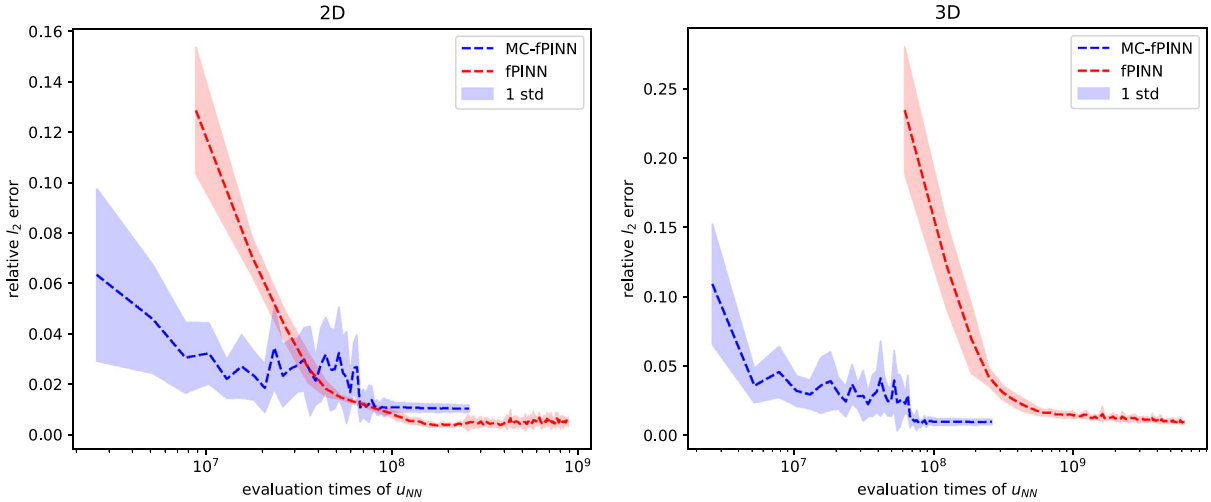

Fig. 6. Efficiency comparison between MC-fPINN and fPINN in terms of accuracy versus computational cost. **Left:** 2D fractional Poisson problem. **Right:** 3D fractional Poisson problem. The Adam optimizer with $M = 10^4$ iterations and mini-batch size $N_f = 128$ is utilized for both fPINN and MC-fPINN.

not practical, and fails to find a satisfactory solution due to the huge memory and time requirements, whereas MC-fPINN can still give an accurate estimate efficiently even for the dimension up to 10. As a conclusion, MC-fPINN alleviates the computational cost greatly compared with fPINN especially for high dimensional problems.

4.2. Inverse problems of fractional advection–diffusion equation (ADE)

We now present the performance of using MC-fPINNs to solve an inverse problem for the space–time fractional differential equation defined by Eq. (1). Specifically, we choose the domain $\Omega = \{x \mid \|x\|_2^2 \leq 1\}$ and consider a fabricated solution $u(x,t) = e^{-t}(1 - \|x\|^2)_+^{1+\frac{\alpha}{2}}$ [30]. According to [44,46], the space-fractional and the time-fractional derivatives of $u(x,t)$ can be computed analytically and thus we can obtain the forcing term

$$\begin{aligned} f(x,t) = &-t^{1-\gamma} E_{1,2-\gamma}(-t)(1-\|x\|^2)^{1+\frac{\alpha}{2}} \\ &+ c2^{\alpha}\Gamma\left(\frac{\alpha}{2}+2\right)\Gamma\left(\frac{\alpha+d}{2}\right)\Gamma\left(\frac{d}{2}\right)^{-1}\left(1-\left(1+\frac{\alpha}{d}\right)\|x\|_2^2\right)e^{-t} + (1+\frac{\alpha}{2}(1-\|x\|^2)^{\frac{\alpha}{2}}(-2x))e^{-t}, \end{aligned} \tag{24}$$

where $E_{a,b}(t)$ is the Mittag-Leffler function defined by $E_{a,b}(t) = \sum_{k=0}^{\infty} \frac{t^k}{\Gamma(ak+b)}$.

Solving the inverse problem with MC-fPINNs has the same flowchart as the forward problem, and the FPDE parameters, which are the targets to be identified, can be optimized together with the DNN parameters during the training process. We now assume that the exact fractional orders α and γ, the diffusion coefficient c, and the flow velocity v are unknown, and aim to use the MC-fPINN method to identify these unknown parameters and the solution $u(x,t)$. In addition, extra measurements from u are assumed to be available for inferring these coefficients. In the domain $\Omega \times \{t = T\}$, we select $N_u = 20, 80, 100$ additional uniformly distributed measurements of u for the 1D/3D/5D problems, respectively.

The "hidden" values of α, γ, c are selected to be 1.5, 0.5 and 0.1, respectively. The true value of v will be specified for 1D/3D/5D problems in Table 1. When setting up the MC-fPINNs, the unknown parameters are coded as "variables" instead of as "constants" so that they will be tuned at the training stage. The initial values of the unknown parameters are taken as $\alpha_0 = 1.7$, $\gamma = 0.9$, $c = 0.5$. The initial value for v is taken from $U[0, 0.1]^d$, where $U[0, 0.1]$ is uniform distribution within $[0, 0.1]$. In practice, these values could be chosen based on reasonable guesses. The sample number m of random instrumental variables in (19) is set to be 30 and the neighborhood radius $r_0 = 0.3$. We use 128 residual points for computing the equation loss for each mini-batch. The neural networks are trained with an Adam optimizer with changing learning rate for 40 000 epochs.

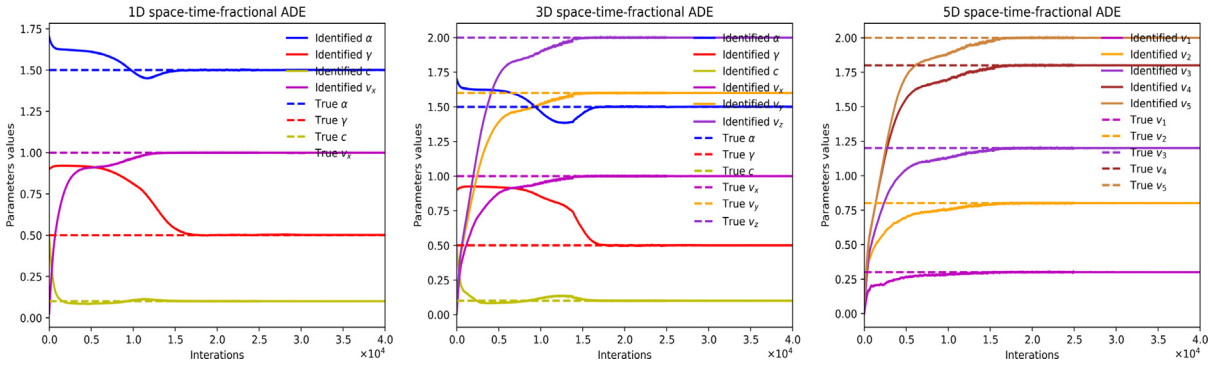

Fig. 7. ADE inverse problem for parameters identification. Parameter evolution as the iteration of optimizer progresses: **Left:** 1D space–time-fractional ADE; **Middle:** 3D space–time-fractional ADE; and **Right:** 5D space–time-fractional ADE.

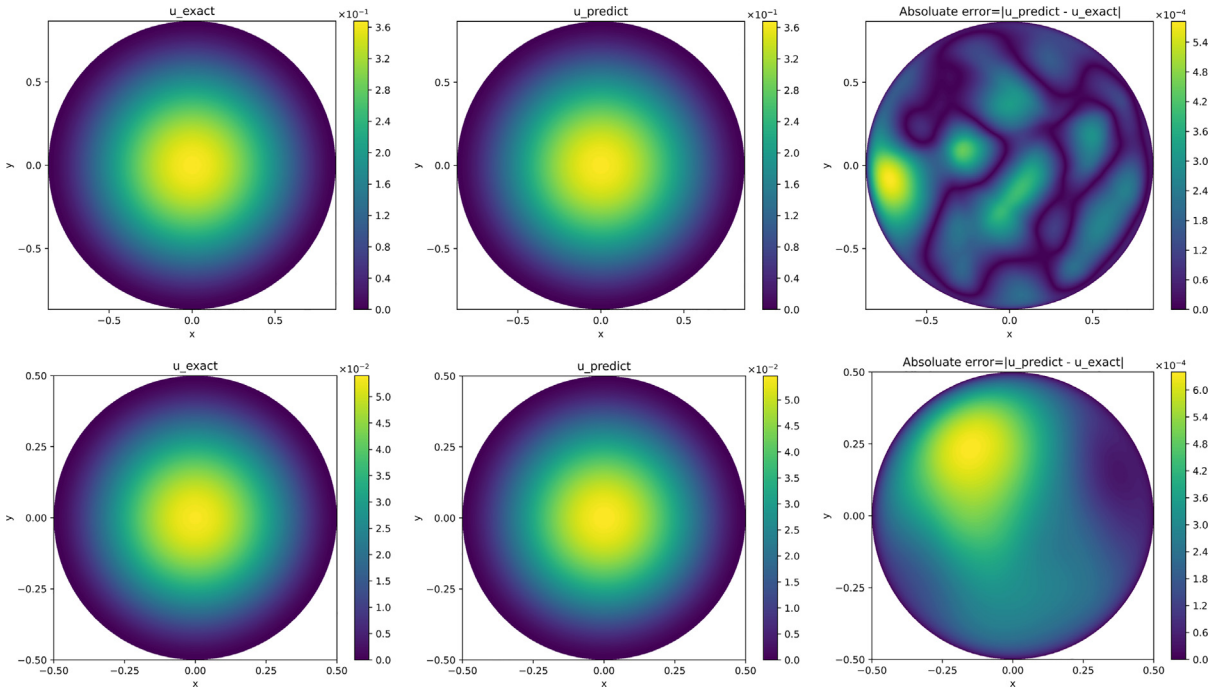

Fig. 8. MC-fPINNs recovery accuracy for inverse space–time-fractional ADE. **Top:** 3D exact solution (left), the MC-fPINNs recovered solution (middle) and the corresponding absolute error at $x_3 = 0.5$, $t = 0.5$ (right). **Bottom:** 5D exact solution (left), the MC-fPINNs recovered solution (middle) and the corresponding absolute error at $x_1 = x_2 = x_3 = 0.5$, $t = 0.5$ (right).

Table 1
Identified parameters and relative errors of the predicted solution field u for 1D/3D/5D inverse ADE problems with synthetic data. The true parameters are $\alpha = 1.5$, $\gamma = 0.5$, $c = 0.1$.

	True parameters (α, γ, c, v)	Identified parameters (α, γ, c, v)	Relative l_2 error
$1D$	1.5, 0.5, 0.1, 1.0	1.50060, 0.50181, 0.09993, 0.99997	5.04×10^{-4}
$3D$	1.5, 0.5, 0.1, 1.0, 1.6, 2.0	1.50115, 0.50005, 0.09981, 1.00000, 1.59993, 2.00000	1.18×10^{-3}
$5D$	1.5, 0.5, 0.1, 0.3, 0.8, 1.2, 1.8, 2.0	1.50188, 0.49736, 0.09968, 0.29976, 0.79998, 1.20005, 1.80015, 2.00016	3.26×10^{-3}

Fig. 7 displays the convergence history of the parameters for the 1D/3D/5D fractional ADEs. We can observe that the inferred values converge to the true values after less than 20 000 training epochs for all the problems. Fig. 8 shows the contour plots of the fabricated solutions, MC-fPINNs recovered solutions, and the absolute errors

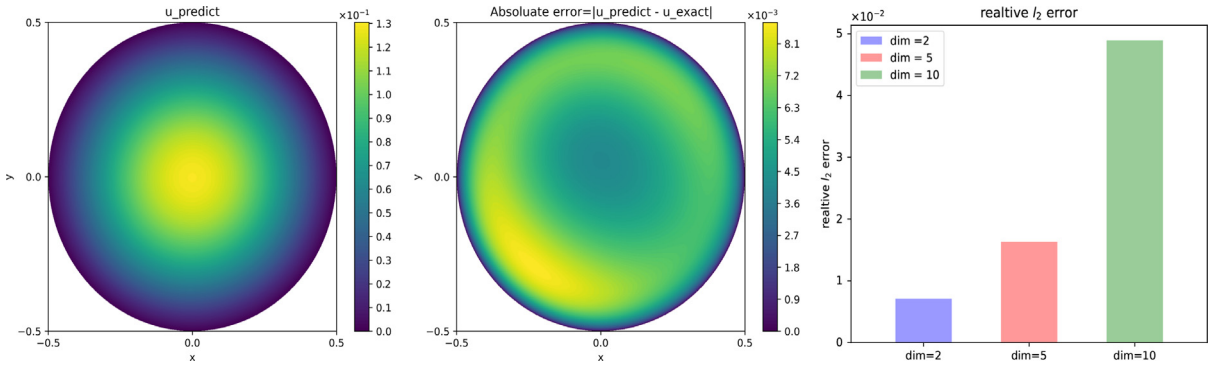

Fig. 9. Fractional diffusion equation with random inputs. **Left:** contour of the MC-fPINNs recovered solution in 5D at $x_3 = x_4 = x_5 = 0.5$; **Middle:** contour of the absolute error between the MC-fPINNs recovered solution and the fabricated solution in 5D; **Right:** Relative l_2 error for 2D, 5D and 10D problem respectively.

of the recovered solutions in comparison with the fabricated solutions, respectively. Table 1 summarizes estimated parameters, including the recovered parameters and the relative l_2 error for 1D/3D/5D problems. We can observe that all the hidden FPDE parameters and the solution field u are well captured, but the error increases as the physical dimension increases.

4.3. Fractional diffusion equation with random inputs

Finally, we consider the following parametric diffusion equation with fractional Laplacian

$$\begin{aligned} (-\Delta)^{\frac{\alpha}{2}} u(x) + \mu u(x) &= f(x, \alpha_0), \quad x \in \Omega \subset \mathbb{R}^d, \\ u(x) &= 0, \qquad\quad x \in \mathbb{R}^d \backslash \Omega, \end{aligned} \tag{25}$$

where α and μ are input random variables in this example, and $\Omega = \{x | \|x\| \leq 1\}$. We assume $\alpha \sim U[0.5, 1.5]$, $\mu \sim U[-0.5, 0.5]$. α_0 is a constant in the forcing term. Our goal for this simulation is to build a DNN surrogate for $u(x|\alpha, \mu)$ given any α and μ. Thus, we can get the statistical approximation of u after the parameters in DNN are fine-tuned. The number of residual points used for computing the equation loss for each mini-batch is taken as 128. The sample number m is set to be 30 and the neighborhood radius is $r_0 = 0.2$. The neural networks are trained with an Adam optimizer with changing leaning rate for 10 000 epochs.

We will demonstrate the performance of the MC-fPINN method for 2D/5D/10D problems. We investigate the accuracy of the MC-fPINNs solution with fixed $\alpha_0 = 1$, $\alpha = \alpha_0$ and $\mu = 0$. This is a special case for the problem considered in Section 4.1. The left and middle plots of Fig. 9 show the contour plot of the MC-fPINNs prediction and the absolute errors of the predicted solutions in comparison with the fabricated solutions for the 5D problem, respectively. The right plot of Fig. 9 plots the relative l_2 error for 2D/5D/10D problems. We can see that the error is increasing for the high dimensional problem, since the same number of residual points are used during the training process.

The surrogate model given by MC-fPINNs can also be applied to parameter identification of the diffusion equation. As an example, we assume that u is defined as in Section 4.1 for $d = 2$, parameters α, μ are unknown, and 5 sensors for u are placed in Ω. Then, we can use the approximate Bayesian computation method [47] to calculate the posterior distribution of parameters for given sensor observations (see Appendix B for details). The approximate posterior densities are shown in Fig. 10.

5. Summary

A deep learning approach for solving forward and inverse problems involving fractional partial differential equations is presented. Using the idea of Monte Carlo quadrature and physics-informed neural networks, we propose a MC-fPINNs method that can flexibly compute the unbiased estimation of the FPDEs-constraint in the loss function during the optimization process of the DNN. This approach substantially mitigates the issue of fast growth in the

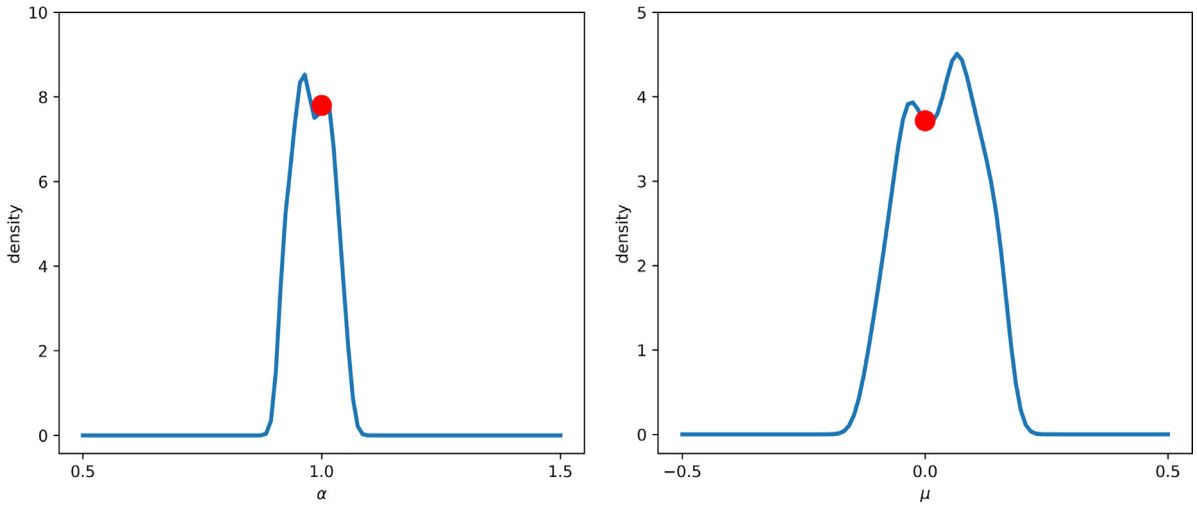

Fig. 10. Approximate posterior distributions of parameters α and μ for $d = 2$, where the sensor positions are $(-0.0120, -0.2170)$, $(0.0321, 0.7628)$, $(0.6677, 0.1095)$, $(0.5411, 0.5840)$, $(-0.2382, -0.7787)$. The density values are calculated by the `gaussian_kde` function of `scipy.stats` module in Python, and red points represent true values of parameters $\alpha = 1, \mu = 0$.

number of auxiliary points, especially for high dimensional problems, which was introduced in [30] to discretize the fractional derivative of the DNN outputs. The performance of the proposed MC-fPINNs is demonstrated for high dimensional integral fractional Laplacian, parametric identification time–space fractional differential equations, and fractional diffusion equations with random inputs. We note here that not all fractional operators can be treated as in the present work, and future work will address this issue as well as extending this approach to nonlocal and peridynamic models. Furthermore, we shall also pursuit the study for fractional PDEs with discontinuous solutions with adaptive sampling strategies.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Data availability

Data will be made available on request.

Acknowledgments

The first author is supported by the NSF of China (under grant numbers 12071301, 11671265) and the Shanghai Municipal Science and Technology Commission (No. 20JC1412500). The second and third author are supported by the NSF of China (No. 12171367) and the Shanghai Municipal Science and Technology Commission (under grant numbers 20JC1413500, 21JC1403700 and 2021SHZDZX0100). The last author is supported by the National Key R&D Program of China (Grant No. 2020YFA0712000), the Strategic Priority Research Program of Chinese Academy of Sciences (Grant No. XDA25010404), and the NSF of China (under grant numbers 11822111, 11688101).

Appendix A. Analysis of the Monte Carlo approximation

In this appendix, we assume $\epsilon = \epsilon_t = 0$ and ignore the rounding error. Then, according to Section 3.2.1,

$$\mathbb{E}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x,t;\theta)\right]\right] = \mathbb{E}\left[\mathcal{L}'_{\mathrm{MC}}\left[u_{NN}(x,t;\theta)\right]\right] = \mathcal{L}\left[u_{NN}(x,t;\theta)\right].$$

A.1. Analysis of $\hat{L}_{\mathrm{equ}}$ in (19)

From the independence of $\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x,t;\theta)\right]$ and $\mathcal{L}'_{\mathrm{MC}}\left[u_{NN}(x,t;\theta)\right]$, we have

$$\begin{aligned}
\mathbb{E}\left[\hat{L}_{\mathrm{equ}}(\theta)\right] &= \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathbb{E}\left[\mathcal{L}_{\mathrm{MC}}[u_{NN}(x_i,t_i;\theta)]\right] - f(x_i,t_i)\right]\cdot\left[\mathbb{E}\left[\mathcal{L}'_{\mathrm{MC}}[u_{NN}(x_i,t_i;\theta)]\right] - f(x_i,t_i)\right] \\
&= \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathcal{L}\left[u_{NN}(x,t;\theta)\right] - f(x_i,t_i)\right]^2 \\
&= L_{\mathrm{equ}}(\theta)
\end{aligned}$$

and

$$\mathbb{E}\left[\frac{\partial}{\partial\theta}\hat{L}_{\mathrm{equ}}(\theta)\right] = \frac{\partial}{\partial\theta}\mathbb{E}\left[\hat{L}_{\mathrm{equ}}(\theta)\right] = \frac{\partial}{\partial\theta}L_{\mathrm{equ}}(\theta).$$

A.2. Analysis of approximation (21)

We define

$$L^s_{\mathrm{equ}}(\theta) = \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x_i,t_i;\theta)\right] - f(x_i,t_i)\right]^2$$

for convenience. Then,

$$\begin{aligned}
\mathbb{E}\left[L^s_{\mathrm{equ}}(\theta)\right] &= \frac{1}{N_f}\sum_{i=1}^{N_f}\left[\mathbb{E}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x_i,t_i;\theta)\right]\right] - f(x_i,t_i)\right]^2 \\
&\quad + \frac{1}{N_f}\sum_{i=1}^{N_f}\mathrm{Var}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x_i,t_i;\theta)\right]\right] \\
&= L_{\mathrm{equ}}(\theta) + \frac{1}{N_f}\sum_{i=1}^{N_f}\mathrm{Var}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x_i,t_i;\theta)\right]\right]
\end{aligned}$$

and

$$\mathbb{E}\left[\frac{\partial}{\partial\theta}L^s_{\mathrm{equ}}(\theta)\right] = \frac{\partial}{\partial\theta}L_{\mathrm{equ}}(\theta) + \frac{1}{N_f}\sum_{i=1}^{N_f}\frac{\partial}{\partial\theta}\mathrm{Var}\left[\mathcal{L}_{\mathrm{MC}}\left[u_{NN}(x_i,t_i;\theta)\right]\right],$$

which imply that estimates provided by (21) are biased in general.

Appendix B. Approximation of the posterior distribution in Section 4.3

In Section 4.3, the posterior distribution of (α, μ) based on the surrogate model is calculated by the approximate Bayesian computation as shown in Algorithm 2, where x_k, u_k denote the location and observation of the kth sensor $(k = 1, \ldots, 5)$, $N_{\text{abc}} = 10^5$ and the tolerance $\epsilon_{\text{abc}} = 2.5 \times 10^{-4}$.

Algorithm 2 Posterior estimation of parameters for given sensor observations by the approximate Bayesian computation

- **1.** Draw N_{abc} parameter points $\{(\alpha_i, \mu_i)\}_{i=1}^{N_{\text{abc}}}$ from the prior $\alpha \sim \mathcal{U}[0.5, 1.5]$, $\mu \sim \mathcal{U}[-0.5, 0.5]$.
- **2.** Calculate the sum of square errors between the sensor observations and those provided by the surrogate model:

$$\text{SSE}_i = \sum_{k=1}^{5} (u(x_k|\alpha_i, \mu_i) - u_k)^2$$

  for $i = 1, \ldots, N_{\text{abc}}$.
- **3.** Accept all parameter points (α_i, μ_i) with $\text{SSE}_i \leq \epsilon_{\text{abc}}$ as samples from the posterior, and approximate the posterior distribution by the histogram of the samples.

References

[1] D.A. Benson, S.W. Wheatcraft, M.M. Meerschaert, Application of a fractional advection-dispersion equation, Water Resour. Res. 36 (6) (2000) 1403–1412.

[2] F. Mainardi, Fractional calculus and waves in linear viscoelasticity: an introduction to mathematical models, World Scientific, 2010.

[3] B.P. Epps, B. Cushman-Roisin, Turbulence modeling via the fractional Laplacian, 2018, arXiv preprint arXiv:1803.05286.

[4] W. Chen, A speculative study of 2/3-order fractional Laplacian modeling of turbulence: Some thoughts and conjectures, Chaos Interdisciplinary J. Nonlinear Sci. 16 (2) (2006) 023126.

[5] F. Song, C. Xu, G.E. Karniadakis, A fractional phase-field model for two-phase flows with tunable sharpness: Algorithms and simulations, Comput. Methods Appl. Mech. Engrg. 305 (2016) 376–404.

[6] A. Lischke, G. Pang, et al., What is the fractional Laplacian? A comparative review with new results, J. Comput. Phys. (2020) 109009.

[7] C. Li, M. Cai, Theory and Numerical Approximations of Fractional Integrals and Derivatives, SIAM, 2019.

[8] W. E, Machine learning and computational mathematics, Commun. Comput. Phys. 28 (2020) 1639–1670.

[9] T. Graepel, Solving noisy linear operator equations by Gaussian processes: Application to ordinary and partial differential equations, in: International Conference on Machine Learning (2003), 2003, pp. 234–241.

[10] S. Särkkä, Linear operators and stochastic partial differential equations in Gaussian process regression, in: International Conference on Artificial Neural Networks (2011), Springer, 2011, pp. 151–158.

[11] I. Bilionis, Probabilistic solvers for partial differential equations, 2016, arXiv preprint arXiv:1607.03526.

[12] M. Raissi, P. Perdikaris, G.E. Karniadakis, Numerical Gaussian processes for time-dependent and nonlinear partial differential equations, SIAM J. Sci. Comput. 40 (1) (2018) A172–A198.

[13] G. Pang, L. Yang, G.E. Karniadakis, Neural-net-induced gaussian processregression for function approximation and PDE solution, J. Comput. Phys. 384 (2019) 270–288.

[14] X. Yang, G. Tartakovsky, A. Tartakovsky, Physics-informed kriging: A physics-informed Gaussian process regression method for data-model convergence, J. Comput. Phys. 395 (2019) 410–431.

[15] T. Lee, I. Bilionis, A.B. Tepole, Propagation of uncertainty in the mechanical and biological response of growing tissues using multi-fidelity Gaussian process regression, Comput. Methods Appl. Mech. Engrg. 359 (2020) 112724.

[16] X. Meng, Z. Wang, D. Fan, M.S. Triantafyllou, G.E. Karniadakis, A fast multi-fidelity method with uncertainty quantification for complex data correlations: Application to vortex-induced vibrations of marine risers, Comput. Methods Appl. Mech. Engrg. 386 (2021) 114212.

[17] I.E. Lagaris, A.C. Likas, D.I. Fotiadis, Artificial neural networks for solving ordinary and partial differential equations, IEEE Trans. Neural Netw. 9 (5) (1998) 987–1000.

[18] I.E. Lagaris, A.C. Likas, D.G. Papageorgiou, Neural-network methods for boundary value problems with irregular boundaries, IEEE Trans. Neural Netw. 11 (5) (2000) 1041–1049.

[19] Y. Khoo, J. Lu, L. Ying, Solving parametric PDE problems with artificial neural networks, European Journal of Applied Mathematics 32 (3) (2021) 421–435.

[20] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics informed deep learning (part I): Data-driven solutions of nonlinear partial differential equations, 2017, arXiv Preprint arXiv:1711.10561.

[21] W. E, B. Yu, The deep ritz method: a deep learning-based numerical algorithm for solving variational problems, Commun. Math. Stat. 6 (2018) 1–12.

[22] Y. Zang, G. Bao, X. Ye, H. Zhou, Weak adversarial networks for high-dimensional partial differential equations, J. Comput. Phys. 411 (2020) 109409.
[23] X. Meng, Z. Li, D. Zhang, G.E. Karniadakis, PPINN: Parareal physics-informed neural network for time-dependent PDEs, Comput. Methods Appl. Mech. Engrg. 370 (2020) 113250.
[24] Y. Liao, P. Ming, Deep nitsche method: Deep ritz method with essential boundary conditions, Commun. Comput. Phys. 29 (2021) 1365–1384.
[25] Y. Kim, Y. Choi, D. Widemann, T. Zohdi, A fast and accurate physics-informed neural network reduced order model with shallow masked autoencoder, J. Comput. Phys. 451 (2022) 110841.
[26] J. Huang, H. Wang, T. Zhou, An augmented Lagrangian deep learning method for variational problems with essential boundary conditions, Commun. Comput. Phys. 31 (2022) 966–986.
[27] M. Raissi, P. Perdikaris, G.E. Karniadakis, Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial dierential equations, J. Comput. Phys. 378 (2019) 686–707.
[28] M. Raissi, Z. Wang, et al., Deep learning of vortex-induced vibrations, J. Fluid Mech. 861 (2019) 119–137.
[29] M. Raissi, A. Yazdani, G.E. Karniadakis, Hidden fluid mechanics: Learning velocity and pressure fields from flow visualizations, Science 367 (6481) (2020) 1026–1030.
[30] G. Pang, L. Lu, G.E. Karniadakis, fPINNs: Fractional physics-informed neural networks, SIAM J. Sci. Comput. 41 (4) (2019) A2603–A2626.
[31] G. Pang, M. D'Elia, M. Parks, G.E. Karniadakis, nPINNs: nonlocal physics-informed neural networks for a parametrized nonlocal universal Laplacian operator. Algorithms and applications, J. Comput. Phys. 422 (2020) 109760.
[32] H. Wang, N. Du., Fast alternating-direction finite difference methods for three dimensional space-fractional diffusion equations, J. Comput. Phys. 258 (2014) 305–318.
[33] M. Ainsworth, C. Glusa, Aspects of an adaptive finite element method for the fractional Laplacian: a priori and a posteriori error estimates, efficient implementation and multigrid solver, Comput. Methods Appl. Mech. Engrg. 327 (2017) 4–35.
[34] Z. Hao, Z. Zhang, R. Du, Fractional centered difference scheme for high-dimensional integral fractional Laplacian, J. Comput. Phys. 424 (2021) 109851.
[35] S. Duo, Y. Zhang, Accurate numerical methods for two and three dimensional integral fractional Laplacian with applications, Comput. Methods Appl. Mech. Engrg. 355 (2019) 639–662.
[36] M. Zhao, H. Wang, A. Cheng, A fast finite difference method for three-dimensional time-dependent space-fractional diffusion equations with fractional derivative boundary conditions, J. Sci. Comput. 74 (2) (2018) 1009–1033.
[37] V. Minden, L. Ying, A simple solver for the fractional Laplacian in multiple dimensions, SIAM J. Sci. Comput. 42 (2) (2020) A878–A900.
[38] J. Burkardt, Y. Wu, Y. Zhang, A unified meshfree pseudospectral method for solving both classical and fractional PDEs, SIAM J. Sci. Comput. 43 (2) (2021) A1389–A1411.
[39] L. Miller, M. Yamamoto, Coefficient inverse problem for a fractional diffusion equation, Inverse Problems 29 (7) (2013) 075013.
[40] W. Rundell, Z. Zhang, Recovering an unknown source in a fractional diffusion problem, J. Comput. Phys. 368 (2018) 299–314.
[41] L. Bottou, O. Bousquet, The tradeoffs of large scale learning, Adv. Neural Inf. Process. Syst. 20 (2007).
[42] P. Mertikopoulos, N. Hallak, A. Kavis, V. Cevher, On the almost sure convergence of stochastic gradient descent in non-convex problems, Adv. Neural Inf. Process. Syst. 33 (2020) 1117–1128.
[43] B. Van Merriënboer, O. Breuleux, A. Bergeron, P. Lamblin, Automatic differentiation in ML: Where we are and where we should be going, Adv. Neural Inf. Process. Syst. 31 (2018).
[44] B. Dyda, Fractional calculus for power functions and eigenvalues of the fractional Laplacian, Fract. Calculus Appl. Anal. 15 (4) (2012) :536–555.
[45] L. Lu, X. Meng, Z. Mao, G.E. Karniadakis, DeepXDE: A deep learning library for solving differential equations, SIAM Rev. 63 (1) (2021) 208–228.
[46] R. Gorenflo, A.A. Kilbas, F. Mainardi, S.V. Rogosin, et al., Mittag-Leffler Functions, Related Topics and Applications, Springer, 2020.
[47] M.A. Beaumont, Approximate Bayesian computation, Annu. Rev. Stat. Appl. 6 (2019) 379–403.